Fakultet informacijskih tehnologija
{elmir.babovic}{denis}{amel.music}{adil.eminagic}{adil}@edu.fit.ba

Datum: 10.03.2026.

Upute za izradu seminarskog rada

U nastavku su navedene upute za prijavu, izradu, predaju i odbranu seminarskog rada iz predmeta Razvoj softvera II.

1. Prijava teme seminarskog rada

- Seminarski rad se primarno izrađuje samostalno. Grupni rad je dozvoljen ukoliko su ispunjeni preduslovi koji se odnose na obim rada i specifičnost teme.
- Tema seminarskog rada može biti iz domene elektronskog poslovanja, ali ne smije predstavljati klasičnu prodaju proizvoda koja se implementira u sklopu nastave na ovom predmetu. Odabrana tema ne može biti u potpunosti zasnovana na funkcionalnostima zajedničkog projekta eProdaja/eCommerce.
- Temu rada predlaže svaki student, bez obzira na to da li seminarski rad izrađuje samostalno ili kao član grupe.
- Kod grupnog rada svaki student predaje zasebnu prijavu u kojoj su opisane sve funkcionalnosti rada, uz jasnu naznaku koji je član tima zadužen za svaku funkcionalnost.
- Ako se radi o grupnom radu, svaki student u svojoj prijavi na početku navodi sebe kao prvog člana tima, a zatim i ostale članove.
- U prijavi je potrebno jasno razdvojiti funkcionalnosti desktop i mobilnog dijela aplikacije.
- Članovi grupe trebaju podjednako učestvovati u svim dijelovima projekta: API-ju, desktop dijelu i mobilnom dijelu aplikacije.
- Tema seminarskog rada predlaže se u formi prijave (.docx datoteka) u kojoj su pobrojane i detaljno opisane sve funkcionalnosti.
- Uz opis funkcionalnosti, prijava mora sadržavati i skice (mockup-e) glavnih dijelova interfejsa koji će biti implementirani.
- Sve skice moraju biti međusobno konzistentne, usklađene s prihvatljivim UI/UX principima i ne smiju prikazivati već gotove, implementirane interfejse.
- Nije potrebno izrađivati skice za dijelove koji su uobičajeni za sve aplikacije, kao što su prijava, resetovanje lozinke, profil i slično.
- Prijava seminarskog rada vrši se putem DL sistema, odnosno putem sekcije Zadaci.
- Tokom perioda u kojima se ne organizuju ispiti, studenti će u okviru navedene sekcije dobijati komentare i eventualne zahtjeve za korekciju prijave.
- Sa implementacijom projekta može se krenuti tek nakon odobrenja prijave.
- Prijave predate nakon definisanih rokova neće biti pregledane.

2. Implementacija seminarskog rada

U nastavku su navedeni obavezni zahtjevi za implementaciju rada.

2.1. Opšti zahtjevi

- U okviru rada moraju biti demonstrirane i specifične implementacijske stavke obrađivane na nastavi. Nastavno osoblje će jasno identificirati stavke na koje je potrebno obratiti pažnju (npr. organizacija

i upravljanje konfiguracijskim podacima, algoritmi korišteni za preporuku i način upravljanja njihovim modelima i sl.).
- Sve funkcionalnosti navedene u prijavi moraju biti implementirane i potpuno funkcionalne.
- Implementacija mora odgovarati opisu iz prijave.

2.2. Desktop dio aplikacije

- Desktop dio predstavlja administrativni dio aplikacije i treba sadržavati modul za izvještavanje.
  - Pregled svih podataka mora uključivati minimalno jedan parametar za pretragu, osim ako to nije opravdano i primjenjivo drugačije (npr. prikaz pet najboljih klijenata).
  - Potrebno je omogućiti minimalno dva izvještaja u .pdf formatu, dostupna za preuzimanje i ispis.
- Aplikacija mora posjedovati forme za upravljanje (CRUD) svim referentnim podacima, kao što su države, gradovi, kategorije, tipovi, statusi i slično, bez obzira na to da li su navedeni u prijavi.

2.3. Mobilni dio aplikacije

- Mobilni dio predstavlja klijentsku aplikaciju i, u zavisnosti od teme i područja za koje se aplikacija razvija, treba sadržavati:
  - pregled uslužnih ili drugih djelatnosti kompanije;
  - eventualno kreiranje narudžbe i pregled historije aktivnosti registrovanih klijenata (prethodne narudžbe, korištene usluge i sl.);
  - uvid u detalje narudžbe i drugih zapisa;
  - pregled i izmjenu profila (lični podaci, slika i dr.);
  - reset lozinke za korisnika;
  - minimalno jednu master-details formu;
  - Ako je u prijavi rada navedena integracija plaćanja (payment), onda ona mora biti implementirana putem stvarnog sandbox okruženja (Stripe, PayPal i sl.), a ne smije biti simulirana. Također, potrebno je implementirati refund logiku preko payment integracije.

2.4. Glavni REST API servis

- Glavni REST API servis mora sadržavati:
  - CRUD operacije za sve glavne i referentne entitete navedene u prijavi;
  - autentifikaciju i autorizaciju korisnika zasnovanu na JWT-u;
  - filtere i pretragu na list endpointima;
  - server-side validaciju korisničkog unosa;
  - logovanje grešaka sa dovoljno informacija za reprodukciju problema.
- U skladu sa funkcionalnostima rada, potrebno je implementirati jednostavniji modul sistema preporuke korištenjem nekog od poznatih algoritama.
- Umjesto algoritma preporuke moguće je implementirati Identity Server radi osiguranja naprednijih oblika autentifikacije korisnika.
- Umjesto algoritma preporuke moguće je implementirati i chatbot koji koristi neki od dostupnih AI API servisa.
- Recommender mora korisniku objašnjavati zbog čega se određeni sadržaj preporučuje - objašnjive preporuke.
- Podaci koji ulaze u recommender, kao što je historija pretraga i slično, moraju se stvarno upisivati u aplikaciji.
- Implementacija recommendera mora odgovarati dokumentaciji. Na primjer, ako dokumentacija opisuje content-based i popularity-based pristup, implementacija mora pratiti odobreni prijedlog.

- Svi signali koji se koriste u bodovanju (scoring) moraju se zaista koristiti; nije prihvatljivo prikupljati podatke poput AvgRating, a zatim ih ignorisati.

3. Nefunkcionalni zahtjevi i standardi implementacije

3.1. Tehnologije i baza podataka

- Za izradu backend dijela aplikacije potrebno je koristiti Visual Studio 2026 i/ili Visual Studio Code uz upotrebu programskog jezika C#.
- Desktop i mobilni dio aplikacije trebaju koristiti posljednje verzije Fluttera i drugih korištenih biblioteka.
- Baza podataka mora sadržavati minimalno 10 tabela, ne uključujući referentne tabele.
- Za izradu baze podataka potrebno je koristiti SQL Server ili drugu relacionu bazu podataka.
- Bazu podataka treba imenovati brojem indeksa, bez prefiksa IB (npr. 180081).
- Svi strani ključevi moraju biti definisani kroz EF konfiguraciju, uz osiguranje referencijalnog integriteta na nivou baze podataka.
- Referentne tabele su pomoćne tabele koje čuvaju statične ili rijetko mijenjane podatke radi standardizacije i smanjenja ponavljanja podataka.
- Primjeri referentnih tabela uključuju:
  - gradove, koji se koriste u adresama korisnika ili narudžbama;
  - kategorije proizvoda, usluga ili drugih entiteta;
  - države za državljanstvo korisnika ili porijeklo proizvoda;
  - UserRole, Role i ostale ASP.NET Identity tabele;
  - many-to-many međutabele koje ne sadrže dodatne atribute od značaja za rad aplikacije.
- Referentne tabele ne pohranjuju podatke vezane za glavne funkcionalnosti aplikacije, već pružaju podršku i standardizaciju za druge tabele.
- Ako u prijavi nije naveden dovoljan broj funkcionalnosti da zadovolji uslov u pogledu broja tabela, potrebno je dodatno implementirati nove funkcionalnosti po vlastitom izboru.
- Sve tabele moraju biti povezane stranim ključevima radi osiguranja referencijalnog integriteta.
- Obavezna polja (npr. ime, prezime) moraju biti označena kao obavezna (NOT NULL) i moraju odgovarati obaveznim poljima na korisničkom interfejsu.
- Baza podataka mora sadržavati sve podatke neophodne za testiranje aplikacije. Podaci se mogu kreirati i prilikom pokretanja aplikacije (migrations).
- Radovi koji sadrže nedovoljan broj zapisa u bazi podataka neće biti detaljnije evaluirani.
- Tokom implementacije aplikacije može se koristiti Code First ili Database First pristup, sa ili bez upotrebe stored procedura.
- Brisanje može biti kaskadno kada to ima smisla, a mora biti onemogućeno kada zapis koriste drugi entiteti. U tom slučaju korisniku treba prikazati jasnu poruku o razlogu. Soft delete purge mora poštovati FK redoslijed (djeca prije roditelja).
- Referentni podaci (grad, kategorija i sl.) moraju biti predstavljeni kao FK prema zasebnim tabelama, a ne kao string polja.
- Modeli, DTO objekti i request objekti moraju biti konzistentni; npr. nije prihvatljivo da se na jednom mjestu koristi string City, a na drugom CityId.
- Seed mora biti konzistentan: HasData seed i runtime seeder ne smiju koristiti različite hash formate. Seed mora sadržavati i slike ako domena koristi slike.
- Tabele moraju imati jasnu ulogu u sistemu; nije dozvoljeno dodavanje suvišnih tabela bez funkcionalne svrhe.

3.2. Mikroservisna arhitektura

- Implementacija mikroservisne arhitekture podrazumijeva postojanje minimalno dva servisa:
  - glavni servis (API) - REST API koji pruža funkcionalnosti klijentskoj aplikaciji, odnosno desktop i mobilnom dijelu;
  - pomoćni servis (npr. worker, consumer, notifier) koji se nalazi u odvojenom projektu/kontejneru, prima poruke iz RabbitMQ-a i izvršava asinhrone zadatke kao što su slanje e-mailova, obrada notifikacija i slično. Pomoćni servis mora izvršavati stvarne zadatke, a ne samo logiranje.
- RabbitMQ se koristi kao posrednik za komunikaciju između servisa.
  - Glavni servis mora slati poruke na RabbitMQ.
  - Pomoćni servis mora slušati poruke i obrađivati ih u pozadini.
- Svi servisi (API, pomoćni servis, RabbitMQ, baza podataka) moraju biti definisani u docker-compose.yml datoteci.
- Svaki servis mora biti funkcionalan i mrežno povezan.

> **Napomene:**
>
> *Korištenje Hangfire-a unutar API servisa ne predstavlja pomoćni mikroservis, jer se izvršava unutar istog kontejnera i procesa i ne zadovoljava uslov dva odvojena servisa.*
>
> *Worker servis mora biti odvojen kontejner. BackgroundService klasa unutar API projekta ne zadovoljava ovaj zahtjev, jer radi u istom procesu kao i API. Potrebno je kreirati zaseban projekat (npr. MojProjekt.Worker) sa vlastitim Dockerfile-om i dodati ga kao poseban servis u docker-compose.yml datoteku.*

- Docker image tagovi moraju biti eksplicitno verzionisani (npr. postgres:16, a ne postgres:latest za ključne servise).
- Potrebno je pripremiti dovoljno log informacija za reprodukciju problema, posebno u situaciji kada aplikacija padne tokom pregleda.

3.3. Konfiguracijski podaci

- Upravljanje konfiguracijskim podacima treba biti centralizirano i vršiti se na jednom mjestu, kako će biti demonstrirano tokom implementacije projekta na vježbama. Nije prihvatljivo fiksirati iste podatke na više lokacija.
- Svi konfiguracijski podaci moraju biti smješteni u konfiguracijske datoteke (.env datoteka) i ne smiju biti hardkodirani unutar izvornog koda niti u appsettings.json datoteci. Pomenuto se posebno odnosi na:
  - RabbitMQ podatke (host, sender i port);
  - SMTP podatke (host, username, password, use ssl i port);
  - Stripe key;
  - JWT key;
  - konekcijski string (connection string);
  - putanju do API-ja;
  - ostale konfiguracijske vrijednosti.
- Adresa API-ja mora biti konfigurabilna korištenjem komande, npr.

```
flutter run --dart-define=API_BASE_URL=http://10.0.2.2:5000
```

Vrijednost se u aplikaciji treba čitati preko String.fromEnvironment('API_BASE_URL').

3.4. Programski kod projekta

- Programski kod koji se ne koristi ne smije biti sastavni dio projekta.
- Kontrole (widgets) za koje ne postoji implementirana funkcionalnost moraju biti uklonjene.
- Kontrole trebaju učitavati samo podatke za koje su namijenjene.
- Magic numbers (npr. statusId = 1, 2, 3) treba zamijeniti enum-ima ili konstantama. Magic stringove za role treba smjestiti u statičku klasu.
- Kontroleri ne smiju sadržavati poslovnu logiku niti direktno pristupati DbContext-u; potrebno je koristiti servisni sloj (kontroler -> servis -> DbContext).
- Servisi koji koriste DbContext moraju biti registrovani kao Scoped, a ne kao Transient.
- Višestruki SaveChangesAsync() pozivi u jednoj operaciji moraju biti unutar eksplicitne transakcije.
- Potrebno je koristiti custom exception tipove (npr. BusinessException, NotFoundException) umjesto generičkog Exception tipa, uz ExceptionFilter ili middleware za mapiranje na HTTP statuse.
- Klijentu se ne smiju izlagati stack trace (za non-development okruženja), interne greške niti infrastrukturni detalji. Greške je potrebno logirati na serveru, a klijentu vraćati standardizovanu poruku.
- Kontroleri ne smiju vraćati Entity objekte direktno; potrebno je koristiti DTO objekte.
- Ne koristiti dynamic tip; koristiti tipizirane DTO objekte.
- Izbjegavati Service-in-Service pozive koji pozivaju SaveChanges(); potrebno se osloniti na dijeljeni DbContext.
- IHttpContextAccessor treba injektovati u servise umjesto manuelnog parsiranja tokena.
- HttpClient treba kreirati preko IHttpClientFactory, a ne direktno preko new HttpClient().
- CORS konfiguraciju treba definisati jednom i eksplicitno navesti dozvoljene origine.
- Duplicirane registracije servisa u Program.cs treba ukloniti; UseCors, AddHttpContextAccessor i AddSwaggerGen ne smiju se pozivati dvaput.

4. Validacija korisničkog unosa

- Aplikacija mora imati potpunu validaciju unosa podataka, uključujući i edit forme.
- Poruke o grešci moraju eksplicitno navoditi format i ograničenja unosa, pružajući korisnicima jasne smjernice za ispravljanje grešaka (npr. „Unesite validan broj transakcijskog računa u formatu: XXXX...“).
- Validacija ne smije zahtijevati nepotrebno uređivanje svih polja.
- Posebno treba voditi računa o sljedećem:
  - ne zahtijevati unos nove lozinke kada se uređuje korisnik;
  - potrebno je postaviti checkbox ili dugme „Izmijeni lozinku“ koje će otvoriti dva polja za unos nove lozinke i potvrde nove lozinke, ili ostaviti dva prazna polja koja će se validirati samo ako se unese nova lozinka;
  - ukoliko administrator uređuje korisnika, ne treba unositi staru lozinku;
  - ukoliko korisnik mijenja vlastitu lozinku, treba potvrditi staru lozinku.
- Forme moraju ispravno reagovati na neispravne unose; false positive poruke nisu prihvatljive.
- Validacijske poruke moraju biti jasno i pregledno prikazane ispod kontrola, a ne unutar input polja ili kao dijalog.
- Polja poput e-mail adrese i telefona moraju imati validaciju formata unosa.
- Nakon uspješnog dodavanja zapisa korisniku treba prikazati adekvatnu poruku o uspjehu, a ne generičku poruku poput „Success“ ili „Bad request“.

5. Autentifikacija i autorizacija

- Aplikacija mora imati implementiranu autentifikaciju korisnika.
- Autentifikacija mora pokrivati sve endpoint-e koji zahtijevaju autorizaciju i ne smije dozvoliti neautoriziran pristup.
  - [Authorize] mora biti postavljen na svim kontrolerima koji pristupaju korisničkim podacima.
  - [AllowAnonymous] je dozvoljen isključivo na login/register endpointima.
  - Write operacije (POST/PUT/DELETE) nikada ne smiju biti otvorene.
- Na admin endpointima obavezna je role-based autorizacija ([Authorize(Roles = "Admin")]), a ne samo [Authorize]. Nazivi rola u seed podacima moraju odgovarati nazivima korištenim u atributima.
- userId se nikada ne prima iz rute ili body-ja za operacije vezane za trenutnog korisnika; uvijek se preuzima iz JWT tokena. Obavezno je provjeriti da korisnik mijenja samo svoje podatke, dok administratoru treba dozvoliti rad nad podacima drugih korisnika.
- Register endpoint ne smije primati role/isAdmin vrijednosti od klijenta; klijent ne smije moći sam sebi dodijeliti privilegije.
- Dev/test endpointi moraju biti zaštićeni env.IsDevelopment() provjerom ili potpuno uklonjeni. Test kontroleri (npr. PayPalTest) ne smiju biti dostupni u produkciji.
- JWT parsiranje mora uključivati validaciju potpisa. Logout mora invalidirati token na serveru, a ne samo lokalno obrisati token.
- Login kredencijali šalju se u body-ju POST zahtjeva, nikada kroz query string parametre.
- Upload i download datoteka moraju imati autorizaciju i ownership provjeru.
- ChatHub i slični real-time endpointi moraju provjeravati membership, npr. da li je korisnik participant u konverzaciji.
- File upload mora validirati MIME tip i magic bytes, a ne samo ekstenziju.
- GitHub README.md datoteka mora sadržavati:
  - sve korake potrebne za pokretanje aplikacije;
  - korisničke podatke za pristup aplikaciji.

Kontekst	Korisničko ime	Lozinka
Desktop verzija	desktop	test
Mobilna verzija	mobile	test
Više korisničkih uloga	nazivUloge	test

6. Korisnički interfejs

- Forme moraju biti pregledne i prilagođene korisnicima, uz poštovanje osnovnih UI/UX principa.
- Dizajn mora osigurati čitljivost, bez prekomjernog korištenja jarkih boja ili transparentnih elemenata koji otežavaju upotrebu.
- Korisničko iskustvo mora biti intuitivno i treba uključivati očekivane funkcionalnosti, poput dugmeta za zatvaranje („X“) u gornjem desnom uglu forme.
- Dropdown liste moraju se puniti podacima iz baze podataka; npr. gradovi moraju biti ponuđeni u padajućem meniju, a ne u textbox-u.

- Forme moraju omogućavati unos podataka putem odgovarajućih kontrola:
  - boolean vrijednosti (true/false) unose se putem checkbox-a ili preklopnika;
  - datumi se unose putem DateTime pickera;
  - stavke za odabir, kao što su gradovi i slični podaci se biraju iz Drop-down liste (padajuće liste);
  - geografske koordinate treba unositi korištenjem alata ili modala za odabir koordinata (odabir na karti ili unosom grada, ulice i sl. pa da API vrati koordinate), a ne direktnim unosom u textbox kontrolu.
- Forme moraju pravilno reagovati nakon uspješnog spašavanja podataka. Ako korisnik nakon snimanja ostaje na formi, polja se trebaju automatski očistiti. Ako poslovni tok predviđa povratak na pregled odnosno historiju zapisa, korisnik treba biti automatski preusmjeren na taj prikaz, pri čemu najnoviji zapis mora biti prikazan na vrhu. Lista zapisa mora automatski prikazati novododani zapis bez dodatnog ručnog osvježavanja.
- Forme ne smiju prikazivati ID vrijednosti iz baze podataka.
- Prikaz podataka iz many-to-many tabela ne smije sadržavati ID-ove niti se smije sastojati isključivo od njih.
- Kada to ima smisla s aspekta poslovne logike, povezani FK objekat treba biti moguće dodati bez napuštanja trenutnog korisničkog toka, npr. preko modala.
- Kontrole na formama ne smiju se međusobno preklapati niti nepotrebno zauzimati veliki dio forme; npr. slike ne bi trebale zauzimati više od 50% prostora forme.

Primjer lošeg dizajna prikazan je na slici ispod. Glavni nedostaci takvog prikaza su neporavnate labele i vrijednosti, što otežava čitanje i čini ekran vizuelno neurednim. Primjerenije rješenje je koristiti dvije kolone - lijevo nazive polja, a desno njihove vrijednosti - ili raspored sa novim redom za svaku stavku.



Slika 1. Primjer nepreglednog rasporeda elemenata na formi

Preporučuje se i upotreba ikonica radi bolje organizacije prikaza podataka, npr.:

- 🗓️ za datume;
- 🚙 za model auta;
- ⛽ za potrošnju goriva;
- 💰 za cijenu.

- Ne dozvoliti otvaranje forme za unos ukoliko preduslovi nisu ispunjeni (npr. FK tabela nema zapisa).
- Na svim relevantnim tabelarnim i list prikazima, za entitete koji imaju sliku, uz naziv entiteta obavezno je prikazati i odgovarajuću sliku (npr. proizvodi, hoteli).
- Za nepovratne akcije (brisanje, plaćanje, slanje narudžbe) obavezan je confirmation dialog.
- Obavezno je dugme „Back“ za navigaciju.
- Nedostupne akcije moraju imati onemogućeno (disabled) stanje uz objašnjenje razloga nedostupnosti (npr. zašto brisanje nije moguće).

7. Glavna poslovna logika sistema

Svaki projekat koji uključuje rezervacije, narudžbe, zakazivanja, posudbe, upise, evidencije ili slične poslovne procese mora implementirati:

- status polje sa definisanim stanjima: Pending -> Confirmed -> Cancelled / Completed; hard delete umjesto promjene statusa smatra se greškom;
- centralizovanu state machine logiku sa jasno definisanim dozvoljenim prelazima; logika ne smije biti raspoređena po kontrolerima;
- provjeru poslovnih pravila pri promjeni statusa (npr. ne odbiti već odobren zahtjev, ne recenzirati prije završetka, ne otkazati plaćeno bez toka za povrat-refund);
- provjeru zauzetosti i preklapanja termina na backendu, a ne samo na frontendu;
- validaciju preduslova na serveru (istekla članarina, kapacitet, dostupnost i dr.);
- unique constraint ili servisnu provjeru za duplikate (isti korisnik + isti termin/trening);
- audit trag: ko je odobrio ili odbio zahtjev, kada je to učinjeno i odgovarajući opis unutar zapisa;
- otkazivanje za korisnika i administratora; odbijanje mora sadržavati razlog i slati notifikaciju;
- ispravno računanje cijena za edge case situacije (npr. trajanje od 25 h, 48 h i sl.).

7.1. Sistem plaćanja

U nastavku su opisane obaveze samo za projekte koji sadrže modul plaćanja.

- Plaćanje mora biti finalizirano na serverskoj strani putem webhook-a ili server-side API verifikacije. Klijent nikada ne smije evidentirati uspješno plaćanje.
- Confirm payment mora biti idempotentan; ako je status već completed, efekti se ne smiju ponovo izvršavati.
- Potrebno je spriječiti višestruko plaćanje iste stavke provjerom da li već postoji otvoren ili završen payment.
- Nakon uspješnog plaćanja, korisnički interfejs mora preći u jasno stanje „Plaćeno“. Dugme za plaćanje ne smije biti vidljivo za već plaćene stavke (IsPaid u response DTO-u).
- Mobilno plaćanje mora biti in-app (Stripe SDK / PaymentSheet ili deep link povratak), a ne preusmjeravanje u vanjski browser bez povratka.
- Ako je refund logika prijavljena u temi, mora biti implementirana. Refund se vrši na osnovu stvarno naplaćenog iznosa, a ne kalkulisane cijene.
- Server mora imati katalog i sam određivati iznos; kupovina ne smije vjerovati klijentu za cijenu.

7.2. Sistemske notifikacije i obavijesti

- Sistemske notifikacije moraju imati:
  - status (pročitano/nepročitano), naslov, tekst i datum/vrijeme;
  - opciju „označi pročitano“;
  - obavezan auto-refresh putem SignalR-a ili polling mehanizma; ručni refresh je neprihvatljiv;
  - notifikacije za sve relevantne događaje (rezervacija, otkazivanje, promjena statusa, plaćanje), a ne samo za jedan događaj.
- Obavijesti (news) moraju sadržavati naslov, tekst, sliku i datum/vrijeme.

8. Dodatni tehnički i projektni standardi

8.1. Tehnički i projektni standard

- Programski kod mora biti čist: bez template ostataka (npr. WeatherForecastController), *mrtvog* koda, neiskorištenih importa, zakomentarisanog koda i duplikata. Duplicirani kod treba eliminisati u skladu s DRY principom; iste klase i servise ne treba držati na dva mjesta.
- Za logiranje koristiti ILogger<T> umjesto Console.WriteLine.
- Potrebno je ispraviti greške u nazivima klasa i datoteka; ne ostavljati copy-paste ostatke iz drugih projekata.
- Nedovršene metode (NotImplementedException) nisu prihvatljive.
- Komentari tipa „razmišljanje naglas“ ne smiju biti prisutni u produkcijskom kodu.
- README.md mora biti uredan i usklađen sa stvarnim stanjem koda.
- Ukloniti zakomentarisane blokove koda.
- Klijentski endpointi ne smiju pozivati nepostojeće backend rute.

8.2. Performanse, paginacija i validacija

- Obavezan je async/await kroz cijeli stack. Nije dozvoljeno koristiti .GetAwaiter().GetResult(), .Wait() ili .Result. Thread.Sleep treba zamijeniti sa await Task.Delay.
- Unutar async metoda potrebno je koristiti async varijante EF poziva (FirstOrDefaultAsync, ToListAsync i sl.).
- SQL Query unutar petlje (N+1 problem) nije prihvatljiv; potrebno je koristiti Include, GroupBy ili batch upite. MapToResponse metode ne smiju ponovo učitavati entitete iz baze.
- Background poslovi trebaju biti realizirani kroz IHostedService/BackgroundService, a ne kroz Task.Run fire-and-forget pristup.
- Upite koji agregiraju podatke treba raditi jednim GroupBy upitom, a ne kroz veći broj odvojenih upita.
- Filtriranje i validaciju cijena treba raditi na nivou baze (Where clause), a ne učitavanjem svih podataka u memoriju pa naknadnim LINQ filtriranjem.
- Podatke koji se učitavaju na svaki request treba keširati. Koristiti IMemoryCache na servisnom nivou, a ne instance-level Dictionary na Transient servisu.
- Environment varijable treba čitati jednom u konstruktoru, a ne pri svakom pozivu.
- Paginacija je obavezna na svakom list endpointu. PageSize mora imati definisan maksimalni limit (npr. 100). Endpointi tipa RetrieveAll bez limita smatraju se greškom za neprihvatanje, jer mogu nepotrebno opteretiti server i predstavljaju potencijalnu osnovu za zloupotrebu ili DoS-slične napade.
- List endpointi ne smiju vraćati velike podatke (PDF blobove, velike base64 slike); treba vraćati samo podatke potrebne za prikaz, dok se detalji prikazuju na posebnom endpointu.

9. Priprema i predaja seminarskog rada

9.1. Opšte napomene

- Neposredno pred svaki ispitni rok bit će definisan termin za predaju seminarskih radova. Ako se organizuju dva termina, period za predaju bit će znatno kraći.
- Na jednom ispitnom terminu nije moguće izvršiti i prijavu teme i predaju seminarskog rada.

- Seminarski rad je potrebno postaviti na vlastiti javni (public) GitHub repozitorij, uz ispravno podešen .gitignore fajl.
- Seminarski radovi pregledaju se isključivo nakon definisanog roka za predaju.
- Prilikom predaje seminarskog rada potrebno je pratiti sljedeće korake:

1. Na DLWMS-u postaviti link na GitHub Release.
   a) Na glavnom meniju odabrati Nastava.
   b) Odabrati godinu studija 3 i predmet Razvoj softvera II.
   c) Odabrati opciju Zadaci.
   d) Na trenutno aktivnom zadatku odabrati link Prijavi temu.
   e) Nakon prijave teme, iz liste Moji radovi odabrati link Detalji.
   f) Postaviti link na tačan GitHub Release te šifru za .env ZIP arhivu.

- Osnovni preduslov za evaluaciju jeste da se rad, odnosno aplikacija, može pokrenuti bez dodatnih intervencija koje podrazumijevaju bilo kakvu modifikaciju programskog koda, dodavanje referenci ili biblioteka, izmjenu linkova, portova, konekcijskih stringova i slično. Aplikacija mora biti stabilna tokom korištenja.

> Napomena: *Pod pokretanjem aplikacije podrazumijeva se mogućnost korištenja i testiranja osnovnih funkcionalnosti aplikacije.*

- Radovi koji budu modifikovani nakon definisanog termina za predaju, bez obzira na prirodu modifikacije, neće biti evaluirani u tom roku.
- Prije predaje obavezno je testirati aplikaciju i u drugim okruženjima, izvan razvojnog okruženja, te otkloniti sve uočene nedostatke.

9.2. Priprema seminarskog rada

Prije predaje seminarskog rada na pregled potrebno je uraditi sljedeće korake:

- Postaviti dokument sa opisom sistema preporuke na git repozitorij seminarskog rada (recommender-dokumentacija.md).
- Napraviti build mobilne Android i desktop Windows aplikacije
- Build fajlovi se **ne postavljaju u Git historiju repozitorija kao commit-ovani binarni fajlovi**, nego se postavljaju kroz sekciju **GitHub Releases** javnog GitHub repozitorija seminarskog rada.
- Na GitHub repozitoriju treba biti kompletan source code aplikacije.
- Za svaki upload za pregled rada potrebno je kreirati **jedan GitHub Release** koji sadrži **jednu ZIP arhivu** sa build fajlovima potrebnim za pregled rada .
  - ZIP arhiva treba sadržavati APK build fajl mobilne aplikacije:
    - `folder-mobilne-app/build/app/outputs/flutter-apk/app-release.apk`
  - ZIP arhiva treba sadržavati EXE build folder windows desktop aplikacije:
    - `folder-desktop-app/build/windows/x64/runner/Release`
  - Preporučeni naziv arhive je **fit-build-20gg-mm-dd.zip.**
  - U GitHub Release **ne postavljati .env fajl** niti druge osjetljive konfiguracijske fajlove.
- Za finalnu predaju potrebno je koristiti **immutable release**. Release je potrebno prvo kreirati kao **draft**, dodati ZIP arhivu, provjeriti ispravnost sadržaja, a zatim ga objaviti.
- Nakon isteka roka za predaju nisu dozvoljene nikakve izmjene repozitorija ni GitHub Release-a, uključujući izmjenu release asseta, ponovno postavljanje ZIP arhive ili objavu novog release-a za isti rok.
- Na DL sistem postavlja se:
  - link na tačnu GitHub release verziju (a ne na kompletan repozitorij);
  - šifra za .env ZIP arhiva.

- Build za pregled rada mora biti dostupan kroz sekciju **Releases** navedenog repozitorija.
- Preporuka da se ne koriste SSL tj. https za pristup API-u, jer self signed certifikati mogu isteći ili biti nevalidni.
- Konfiguracijski fajlovi sa tajnama (connection stringovi, tokeni, secret ključevi itd – obično .env fajl) treba zamijeniti sa ZIP verzijom (koristiti šifru „fit“ ili random šifru). Zamjenski fajl se treba nalaziti u istom folderu kao originalni fajl, npr. „.env-tajne.zip“ umjesto „.env“ fajla.

9.2.1. Upute za Android aplikaciju

- Adresa za Web API treba biti **10.0.2.2**, a radi se o standardnoj adresi za Google Android Emulator AVD. Očistiti stare build fajlove.

```
flutter clean
```

- Uradite build mobilne aplikacije

```
flutter build apk --release
```

- Prethodni korak bi trebao generisati instalacijski .apk fajl na putanji

```
moj-folder-mobilne-app/build/app/outputs/flutter-apk/app-release.apk
```

- Provjeriti ispravnost rada Android aplikacije u AVD prateći sljedeće korake:
  - Obrišite staru verziju aplikacije u AVD
  - Prevucite .apk fajl u AVD da biste instalirali aplikaciju
  - Pokrenite aplikaciju nakon instalacije i provjerite ispravnost rada.

9.2.2. Upute za Windows aplikaciju

- Adresa za Web API treba biti **localhost**
- Očistiti stare build fajlove

```
flutter clean
```

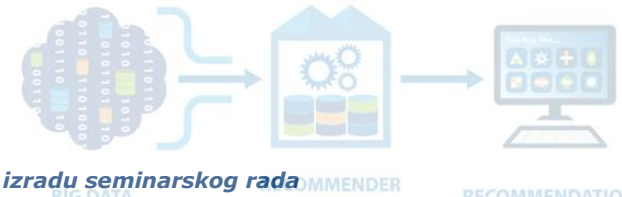
- Napravite build Windows aplikacije

```
flutter build windows --release
```

- Prethodni korak bi trebao generisati .exe fajl na putanji

```
moj-folder-desktop-app/build/windows/x64/runner/Release/
```

- Pokretanjem prethodno generisanog .exe fajla provjeriti ispravnost rada windows aplikacije



9.2.3. Upute za upload GitHub Release-a

- Nakon što pripremite ZIP arhivu sa build fajlovima, otvorite javni GitHub repozitorij seminarskog rada.
- Za finalnu predaju seminarskog rada obavezno je koristiti GitHub opciju **Immutable Release**.
- Immutable Release označava release kod kojeg, nakon objave, nije moguće mijenjati ni brisati release assete, niti mijenjati povezani Git tag. Ova opcija se koristi kako bi objavljeni build fajlovi ostali trajno vezani za stanje rada u trenutku predaje. Uključivanje ove opcije važi samo za **buduće** release-ove. Release-ovi kreirani prije uključivanja ove opcije ne postaju automatski immutable.
    - U repozitoriju odaberite karticu **Settings**.
    - Spustite se do sekcije **Releases**.
    - Uključite opciju Enable release immutability.
    - Nakon uključivanja ove opcije, immutability će važiti za sve naredne release-ove koji budu objavljeni iz tog repozitorija.
    - Link na dodatne informacije: https://docs.github.com/en/code-security/how-tos/secure-your-supply-chain/establish-provenance-and-integrity/preventing-changes-to-your-releases
- Nakon uključivanja policy za Immutable Release, otvorite sekciju **Releases**, a zatim odaberite opciju **Draft a new release**.
- Release je potrebno prvo kreirati kao **draft**.
- U polju za tag unesite oznaku release-a. Preporuka je koristiti naziv usklađen sa datumom build-a, npr. predaja-20gg-mm-dd.
- U polje **Release title** unesite naziv release-a, po mogućnosti istog formata kao tag ili jasno prepoznatljiv naziv koji odgovara roku predaje.
- U dijelu za dodavanje binarnih fajlova postavite ZIP arhivu sa build fajlovima, npr. **fit-build-20gg-mm-dd.zip.**
- Release automatski uključuje i GitHub-ove arhive izvornog koda vezane za taj release (**Source code.zip** i **Source code.tar.gz**).
- U release ne postavljati .env fajl niti druge osjetljive konfiguracijske fajlove.
- Na **draft release** dodajte ZIP arhivu sa build fajlovima i provjerite da je upload završen.
- Tek nakon toga kliknite **Publish release**.
- Nakon objave *immutable release*-a, na GitHub-u više nije moguće mijenjati ni brisati release assete niti mijenjati povezani tag.
- Na DL sistem postavlja se šifra za .env ZIP arhivu.
- Na DL sistem postavlja se link na tačan GitHub Release, a ne link na glavni GitHub repozitorij.
    - Link na tačan GitHub Release treba sadržavati oznaku konkretnog release-a odnosno taga.
    - Opšti oblik takvog linka je:
        - https://github.com/<vlasnik >/<naziv-repo>/releases/tag/<oznaka-release-a>
    - Primjer stvarnog GitHub Release linka:
        - https://github.com/obsproject/obs-studio/releases/tag/31.1.0
    - Ne postavljati link oblika releases/latest, jer taj link uvijek pokazuje na trenutno najnoviji release i kasnije može voditi na drugu verziju, što nije pogodno za evidenciju predaje

9.3. Ocjena seminarskog rada

- Radovi koji na bilo koji način odstupaju od pravila definisanih u ovom dokumentu ili prijavljuju greške pri korištenju osnovnih funkcionalnosti neće biti prihvaćeni. Ovo se posebno odnosi na radove koji, i nakon korekcija, zadržavaju nedostatke na koje je studentu već ranije ukazano.
- Nakon pregleda, lista prihvaćenih seminarskih radova bit će objavljena putem obavijesti na DL sistemu.
- Studenti kojima seminarski rad ne bude prihvaćen dobit će kraći komentar sa glavnim razlozima neprihvatanja. Ako rad ima značajnije nedostatke, neće biti detaljnije pregledan, a student je dužan ispraviti iste ili slične nedostatke u cijelom radu.
- Ako određeni seminarski rad ne bude prihvaćen više od pet puta, nastavno osoblje, u zavisnosti od razloga neprihvatanja, zadržava pravo njegovog poništavanja. U tom slučaju student je dužan definisati novu temu i ponoviti kompletnu proceduru opisanu ovim dokumentom.

Rok za završetak svih obaveza na predmetu je posljednji apsolventski rok, odnosno april naredne akademske godine. Studenti koji ne završe sve obaveze na predmetu dužni su svoje obaveze realizovati prema uputama za novu akademsku godinu.

10. Odbrana seminarskog rada

1. Nakon prihvatanja aplikacije, odnosno seminarskog rada, studentima će biti omogućen pristup odbrani.
2. Tokom odbrane rada studenti dobijaju zadatke u okviru kojih se zahtijeva implementacija i/ili opis određenih funkcionalnosti (API, Flutter desktop i mobilni dio) u okviru vlastitog ili template projekta.
3. Ako je seminarski rad rađen u grupi, svi članovi grupe moraju biti detaljno upoznati sa svim dijelovima aplikacije, bez obzira na funkcionalnosti koje su pojedinačno implementirali.
4. Studentima koji više od tri puta budu neuspješni u odbrani seminarskog rada nastavno osoblje može poništiti postojeću temu i dodijeliti novu.

11. Završne napomene

- U okviru nastave bit će kreirana aplikacija eProdaja/eCommerce sa svim prethodno navedenim komponentama. Studenti će imati pristup video materijalima dostupnim putem FIT servisa.
- Po potrebi, dodatna pojašnjenja ili dopune procedura koje se odnose na izradu seminarskih radova bit će date tokom nastave i objavljene u okviru zasebnih dokumenata ili obavijesti na predmetu.

Dodatak A - Preporuke

Ovaj dodatak sadrži listu čestih tehničkih grešaka i anti-pattern-a uočenih u studentskim projektima, zajedno sa preporukama za ispravnu implementaciju.

A.1. RabbitMQ i messaging

- Za async lambda handlere u consumer-u koristiti AsyncEventingBasicConsumer, a ne EventingBasicConsumer.
- Ne kreirati novu RabbitMQ konekciju pri svakom publish pozivu; koristiti singleton konekciju ili connection pool.

- Ako consumer ne uspije obraditi poruku, implementirati retry logiku sa eksponencijalnim backoff-om (npr. 1 s -> 2 s -> 4 s -> 8 s). Greške ne smiju biti ignorisane u praznim catch blokovima.
- Error handling je obavezan; worker ne smije tiho postati nedostupan bez logiranja razloga.

A.2. Flutter i mobilni kod

- Serijske HTTP pozive koji se mogu paralelizirati treba smjestiti u Future.wait() umjesto sekvencijalnog await-a jednog po jednog.
- Base64 dekodiranje slika ne treba raditi u build metodi koja se poziva na svaki frame; dekodiranje treba izvršiti jednom i rezultat keširati, ili koristiti URL umjesto base64 stringa.
- Deep link handling mora pokriti i initial link scenario i resumed app scenario.
- Frontend mora pravilno obraditi HTTP 401 odgovor (redirect na login ili refresh token mehanizam); istekle tokene ne treba ignorisati.
- Generičke poruke o grešci treba zamijeniti specifičnim porukama. _handleResponse ne smije prikrivati backend validacijske poruke, već ih treba proslijediti korisniku.
- Datoteke sa mock/test nazivima treba preimenovati ako sadrže produkcijski kod.

A.3. Kriptografija i sigurnost lozinki

- Za generisanje kodova, tokena, referral kodova i sličnih sigurnosnih vrijednosti koristiti RandomNumberGenerator iz System.Security.Cryptography, a ne System.Random.
- Za hashiranje lozinki koristiti bcrypt, Argon2 ili PBKDF2. Nije prihvatljivo koristiti SHA256 bez salta, HMAC-SHA512 niti custom sheme.
- Reset tokeni moraju imati definisan istek (ExpiryTime), a reset kodovi se ne smiju čuvati u plain text formatu.

A.4. Datum i vrijeme

- Potrebno je standardizovati DateTime.UtcNow u cijeloj aplikaciji. Miješanje DateTime.Now i DateTime.UtcNow dovodi do nekonzistentnosti u vremenskim zapisima, posebno u Docker okruženju.

A.5. Resursi i dispose

- Svi disposable objekti (Stream, HttpResponseMessage, DbConnection i sl.) moraju biti unutar using statement-a ili using deklaracije kako bi se osiguralo pravovremeno oslobađanje resursa.